

ROUTING • AI GATEWAYS • CASCADES • EVALUATION • INFERENCE • ADAPTIVE SELECTION

Model Routing, AI Gateways & Agentic Systems for Cost, Quality and Performance

A Practical Guide to Building Intelligent Multi-Model AI Systems with Routing, Evaluation, Inference Optimization, and Adaptive Model Selection

AI ENGINEERING INSIDER • aiengineeringinsider.com



Model Routing, AI Gateways & Agentic Systems for Cost, Quality and Performance

A Practical Guide to Building Intelligent Multi-Model AI Systems with Routing, Evaluation, Inference Optimization, and Adaptive Model Selection

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

First edition, 2026. Published by AI Engineering Insider.

Web: aiengineeringinsider.com Newsletter: aiengineeringinsider.substack.com/subscribe

Disclaimer. The information in this book is provided on an “as is” basis, without warranty of any kind. Product names, logos, and brands, including OpenAI, Anthropic, Google, Meta, DeepSeek, Qwen, Alibaba, Mistral, NVIDIA, Ollama, LiteLLM, OpenRouter, Portkey, Kong, vLLM, SGLang, TensorRT-LLM, llama.cpp, LangChain, LangGraph, PostgreSQL, pgvector, Neo4j, Redis, OpenTelemetry, Prometheus, Grafana, Langfuse, Next.js, Vercel, FastAPI, and Pydantic, are the property of their respective owners and are used for identification purposes only; their use does not imply endorsement. This book is an independent educational work and is not affiliated with or endorsed by any model provider, framework maintainer, or standards body named within.

On figures that change. Model names, context limits, prices, benchmark scores, rate limits, and framework capabilities change frequently, and in this field they change monthly. To keep the arguments durable, the model catalog used throughout this book is deliberately *synthetic*: five providers and twelve deployments whose prices, latencies, and quality vectors are representative of the market’s shape rather than a snapshot of any vendor’s price list. Every measured figure comes from a local run of the companion project against that catalog. Reproduce each number on your own traffic and your own contracts before relying on it in a design document or a purchasing decision. The publisher assumes no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

On the security material. Chapters 10 and 14 describe prompt injection, tool-authorisation abuse, and data-exfiltration paths at the level required to defend against them. The material is defensive in orientation and is intended solely for testing and hardening systems you are authorised to test.

Companion project. The code discussed throughout is [model-routing-labs](https://github.com/lamhotsiagian/model-routing-labs), a runnable FastAPI + Next.js routing platform designed to be operated locally, with or without a model server. Every chapter’s “Testing It from the UI” section refers to a lab route in that application.

Code availability. The complete companion project is published at <https://github.com/lamhotsiagian/model-routing-labs>. Every listing in this book is an excerpt from that repository, and every measurement reported here can be reproduced by running it.

Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

There is a decision in every AI system that almost nobody designs, and it is the decision that sets the bill.

Someone picks a model. It goes in a config file, or worse, in a constant near the top of a module. Traffic arrives that is nothing like the traffic the choice was made for: greetings and contract analysis and arithmetic and code review, all going to the same endpoint at the same price. The system works. It is also, quietly, paying frontier prices for “thanks, that helped” and sending a legal question to a model that has never been evaluated on legal questions, and nobody notices either until the invoice or the incident.

Model routing is the discipline of making that decision explicitly, per request, with the evidence available at the time. It is not a cost hack. Done properly it is a control plane: it decides which vendor sees which customer’s data, what happens when a provider rate-limits you at 09:00 on a Monday, how a new model gets 1% of traffic before it gets all of it, and what the system does when the answer it got back is not good enough. Cost is the part that gets the meeting; availability and governance are the parts that get the architecture review.

The questions that distinguish engineers in this area are not definitional. They are: *your router sends 30% more traffic to the large model this week and no deploy happened, so investigate. Your cheap route is now your most expensive line item, explain how. A provider returns 429s for eleven minutes and your p99 triples for an hour afterwards, diagnose. A model you depend on is deprecated in thirty days and you have 140 prompts tuned to it, plan.* Those questions have real answers, and the answers are engineering rather than vocabulary.

Each chapter here builds the mental model for one layer of a routing system, states the trade-offs where decisions actually branch, gives production-grade code from a project you can run locally, and then works five interview questions at the depth a senior interview expects: system design, production operations, debugging, technical mechanism, and case studies drawn from failure modes that recur across real deployments.

The perspective throughout is a system designer’s. Research asks which model is better. Production asks which model is better *for this request, under this latency SLO, at this cost ceiling, with this data classification, given that this provider is currently degraded*, and the answer changes with all five. A cascade is not better than a single model; it is what you use when you have a cheap verifier and a wide difficulty distribution. An LLM router is not better than rules; it is what you use for the residual your rules cannot classify, on the traffic that can afford the latency. Chapter after chapter, the argument is not that one approach wins, but that you should be able to say which one wins for you, and show the number.

More practitioner deep dives: <https://aiengineeringinsider.substack.com/subscribe>

AI Engineering Insider
2026

About This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This book is a system-design-level guide to multi-model AI systems: how a request becomes a routing decision, how that decision becomes an execution plan with fallbacks and escalation, how the result is priced, verified, observed, and governed, and how the whole policy is learned from feedback rather than guessed. It is written for AI engineers, model routing and multi-model systems engineers, machine learning engineers, agentic-AI engineers, software engineers moving into AI, and AI architects who are past the tutorials and need the patterns, trade-offs, failure modes, and runnable code that show up in production and in senior interviews.

The Design Contract

Principle	What it means in practice
Filter, then score	Capability, context, residency, and data class are binary gates. A model that cannot serve the request must never appear in the scoring function, however cheap it is.
Cost per <i>successful</i> task	The only cost metric safe to optimise. Cost per request and quality point in opposite directions often enough to invert decisions.
The router must be cheaper than the route	A classifier that costs more than the model it saves has negative expected value. Order strategies by cost and stop at the first confident one.
Escalation and failover are different systems	One is a quality mechanism triggered by a verdict; the other is an availability mechanism triggered by an exception. Sharing a code path means spending frontier money on a capacity blip.
Routing is a data-egress decision	Choosing a model chooses a legal entity, a jurisdiction, and a retention policy. Classify the payload before you route it, never after.
Confidence must be calibrated	An uncalibrated 0.9 is worth about 0.72. Thresholds set on raw scores escalate the wrong requests.
Diversity is the point of a fallback	Three deployments behind one vendor are one point of failure written three times.
The decision must be replayable	Persist the candidate vector, not the winner. Every incident review starts with “why did it pick that?” and a scalar cannot answer it.
Measure against the oracle	A router that always picks the biggest model scores 100% on answer quality. Only cost regret against the cheapest acceptable model evaluates a <i>router</i> .
Explore on purpose, bounded	A policy that never tries a new model cannot discover that it is better and cheaper. Spend a known fraction of traffic learning.

How This Volume Is Organised

Ch.	Title	Main focus
1	Model Routing Fundamentals	Why one model cannot serve a mixed workload; selection versus routing; specialisation, cascades, ensembles, escalation, and fallback as four distinct patterns; routing against Mixture-of-Experts; the six-stage reference router; the design principles the rest of the book applies.
2	LLM Capabilities, Model Profiling & Selection	The capability matrix as binary gates rather than scores; reasoning, coding, embedding, vision, audio, and multimodal families; open-weight against proprietary; context, tool calling, and structured output; measuring quality, cost, latency, throughput, reliability, and risk; building a registry that survives a deprecation.
3	Routing Strategies & Decision Engines	The strategy chain ordered by cost: metadata, rules, semantic centroids, classifiers, and an LLM router of last resort; capability, domain, tier, geography, and policy routing; margin as confidence; the decision engine and its rejection map.
4	Complexity, Confidence & Adaptive Model Selection	Estimating instance difficulty rather than class difficulty; the cost-ordered estimator cascade; logprob, agreement, and kNN confidence; calibration and expected calibration error; abstention against escalation; early exit and adaptive computation; routing with conversation state.
5	Model Cascades, Fallbacks & Reliability	Cascade economics and when the verifier pays for itself; escalation policy against failover policy; the closed error vocabulary and one correct response per kind; retries with decorrelated jitter, breakers, bulkheads, backpressure, and load shedding; graceful degradation and loop prevention.
6	Cost, Latency & Multi-Objective Routing	Token economics including cache pricing and GPU amortisation; TTFT, TPOT, ITL, and why p95 latency is a distribution not a number; latency budgets and SLA-aware routing; utility functions, normalisation, Pareto fronts, and the budget ledger that degrades rather than fails.
7	Multi-Model & Multi-Provider Architecture	The adapter seam and the error dialect problem; unified APIs, gateways, and what leaks through them; health as three signals rather than a ping; least-outstanding-requests load balancing; hybrid local and cloud inference; version, lifecycle, and deprecation management.
8	AI Gateway, Model Registry & Routing Infrastructure	The pipeline and why its order is load-bearing; request normalisation; the model, provider, and capability registries; policy and routing engines; cost tracker, rate limiter, retry and fallback managers; exact, semantic, and prefix caches; canary and A/B routing; LiteLLM, OpenRouter, Portkey, and cloud gateways.
9	Agentic AI Model Routing	Routing per role rather than per agent; planner, executor, critic, judge, and why the executor decides the bill; supervisor and hierarchical architectures; agent-to-agent handoffs that carry constraints; capability matching, dynamic agent selection, hop budgets, and the arithmetic of routing loops.
10	Routing Across RAG, Tools, MCP & Multimodal Systems	Query shape as the retrieval-routing signal; retriever, index, and reranker routes; reciprocal rank fusion across incomparable channels; tool routing by retrieval and permission; MCP discovery, versioning, and trust; modality pipelines against single multimodal models.

continued on the next page

Ch.	Title	Main focus
11	Model Ensembles, Judges & Collaborative Intelligence	The three things an ensemble can buy and the gate that decides whether it buys any; majority and weighted voting; error correlation and why the fifth member is free of benefit; LLM-as-a-judge with position, self-preference, and length bias handled; debate and Mixture-of-Agents.
12	Inference Engineering for Multi-Model Systems	Where the priced-black-box abstraction breaks; continuous batching and the latency knee; KV-cache capacity as the real limit; PagedAttention, prefix and semantic caching; quantisation and speculative decoding as routing inputs; load-aware and capacity-aware placement; capacity planning.
13	Evaluation, Observability & Routing Intelligence	Why answer quality cannot evaluate a router; the oracle set and cost regret; overrouting against underrouting; utilisation, escalation, and fallback rates; counterfactual cost; OpenTelemetry spans and GenAI conventions; drift detection by population stability index; interleaved online evaluation.
14	Security, Governance & Enterprise Model Routing	Payload classification before routing; PII-aware and residency-aware routes; provider trust boundaries and retention posture; prompt injection as an authorisation problem; RBAC and ABAC over routing decisions; model risk classification, approval workflow, inventory, and audit.
15	Adaptive Routing, Bandits, RL & Production System Design	Feedback loops and reward design; epsilon-greedy, UCB1, Thompson sampling, and LinUCB; exploration under a cost ceiling; learned and neural routing; Mixture-of-Experts as token-level routing with the same load-balancing problem; the end-to-end 100M-request architecture, failure recovery, capacity planning, and case studies.

The Companion Project

Every chapter ends with production code from [model-routing-labs](https://github.com/lamhotsiagian/model-routing-labs) and a section showing how to exercise it from the browser. The stack is FastAPI and Pydantic for the gateway, a JSON model catalog for the registry, Next.js for the labs UI, and a deterministic provider simulator so that every cost, latency, cascade, and failover behaviour in this book is reproducible on a laptop with no vendor keys and no GPU. Set `offline: false` in any lab's control panel to drive real local models through Ollama instead.

Code availability. <https://github.com/lamhotsiagian/model-routing-labs>

Lab	Module	What you can observe
/labs/1	routing/engine.py	A six-request workload routed against a single-model baseline: per-request cost, quality, and the aggregate trade.
/labs/2	routing/registry.py	The capability matrix, the eligible set, and the reason every excluded model was excluded.
/labs/3	routing/strategies.py	The strategy chain with per-strategy confidence, reason, and microsecond timings.
/labs/4	routing/complexity.py	Difficulty features, agreement confidence, and an ECE that falls from 0.150 to 0.041 after calibration.
/labs/5	routing/cascade.py	Escalation against failover under injected 429s, timeouts, and 500s, with live breaker state.
/labs/6	routing/economics.py	Weight sliders, the ranked candidate set with its component vector, and the Pareto front.
/labs/7	routing/health.py	A model that passes a liveness ping while being eight times slower, and the migration ramp that rolls itself back.
/labs/8	routing/gateway.py	The whole pipeline: policy, classification, shedding, routing, caches, quota, execution, audit.
/labs/9	routing/agents.py	Per-role model assignment, the cost counterfactual against one model, and the loop guard firing.
/labs/10	routing/rag_routing.py	Query shape, channel selection, RRF fusion, reranker decision, and permission-filtered tool retrieval.
/labs/11	routing/ensemble.py	Votes, dissent, the ensemble gate, and cost-per-success against member count and correlation.
/labs/12	routing/serving.py	KV utilisation, prefix warmth, the batching knee, and a capacity plan.
/labs/13	routing/evaluation.py	The oracle report: accuracy, overroute, underroute, cost regret, confusion, and PSI drift.
/labs/14	routing/security.py	Detection, typed redaction, tier escalation, eligible-model collapse, and the audit trail.
/labs/15	routing/adaptive.py	Three bandits on the same traffic, a contextual bandit that learns a per-difficulty policy, and MoE expert load.

How to Use This Book

Read the overview and the diagram of a chapter to build the mental model, run the lab to see the behaviour, then work the five questions before reading the answers. The questions are the point; the chapter body exists to make them answerable.

If you are preparing on a deadline, the highest-yield path is Chapters 1, 3, 5, and 6: the routing taxonomy, the strategy chain, the escalation-versus-failover distinction, and multi-objective scoring are where the majority of depth questions land. If you are interviewing for a platform or infrastructure role, add Chapters 7, 8, and 12. If the role is agent-facing, add Chapters 9 and 10. If it is research-adjacent, add Chapters 11 and 15.

Contents

Preface	i
About This Book	ii
Part I Foundations	1
1 Model Routing Fundamentals	2
1.1 Overview	2
1.2 Why Single-Model Architectures Fail	2
1.3 Model Selection Versus Model Routing	4
1.4 Multi-Model Systems and the Five Composition Patterns	5
1.5 Model Routing Versus Mixture-of-Experts	6
1.6 Model Routing in Agentic AI	7
1.7 Core Architecture of a Model Router	7
1.8 Model Routing Design Principles	9
1.9 Production Code: The Decision Object	10
1.10 Testing It from the UI	12
1.11 Interview Questions	13
1.12 Answers	14
2 LLM Capabilities, Model Profiling & Selection	19
2.1 Overview	19
2.2 The Capability Matrix	19
2.3 Model Families and What They Are For	20
2.4 The Seven Properties of a Profile	23
2.5 Measuring the Profile Honestly	25
2.6 Registry Architecture	26
2.7 Model Lifecycle and Deprecation	29
2.8 Testing It from the UI	29
2.9 Interview Questions	31
2.10 Answers	32
Part II The Decision	37
3 Routing Strategies & Decision Engines	38
3.1 Overview	38
3.2 The Economics of Classification	38
3.3 The Strategy Families	39
3.4 Policy Strategies: Constraints, Not Choices	44

3.5	The Chain and Its Fall-Through	45
3.6	Tuning Thresholds	46
3.7	From Classification to Route	47
3.8	Testing It from the UI	47
3.9	Interview Questions	49
3.10	Answers	50
4	Complexity, Confidence & Adaptive Model Selection	54
4.1	Overview	54
4.2	Static Complexity: Features That Correlate With Escalation	55
4.3	Reference-Set Complexity: kNN Against Your Own Traffic	56
4.4	The Cost-Ordered Estimator Cascade	58
4.5	Confidence: Three Signals, One Contract	59
4.6	Calibration: Making Confidence Mean Something	61
4.7	Abstention: The Option Everyone Forgets	63
4.8	From Band to Tier	64
4.9	Early Exit and Adaptive Computation	64
4.10	Routing With Conversation State	65
	Case Study: Sawtooth Routing in a Support Assistant	65
4.11	Testing It from the UI	66
4.12	Interview Questions	67
4.13	Answers	68
Part III	Reliability and Economics	74
5	Model Cascades, Fallbacks & Reliability	75
5.1	Overview	75
5.2	Cascade Economics: When the Verifier Pays for Itself	76
5.3	Verifiers: The Component That Decides Everything	76
5.4	The Escalation Ladder	78
5.5	Failover: Sideways, Not Up	79
5.6	The Error Vocabulary	80
5.7	Retry: Decorrelated Jitter and Nothing Else	81
5.8	Circuit Breakers, Bulkheads, and Backpressure	82
5.9	The Executor: One Loop, Two Mechanisms	85
5.10	Graceful Degradation	86
5.11	Testing It from the UI	87
5.12	Interview Questions	89
5.13	Answers	90
6	Cost, Latency & Multi-Objective Routing	96
6.1	Overview	96
6.2	Token Economics	96
6.3	Latency Is Three Numbers, Not One	99
6.4	The Utility Function	101
6.5	The Pareto Front: What Scalarisation Hides	103
6.6	Budgets That Degrade Instead of Failing	104

6.7	Testing It from the UI	105
6.8	Interview Questions	107
6.9	Answers	108
Part IV Platform		114
7	Multi-Model & Multi-Provider Architecture	115
7.1	Overview	115
7.2	The Adapter Seam	115
7.3	Health Is Three Signals	118
7.4	Load Balancing Across Equivalent Deployments	120
7.5	Hybrid Local and Cloud Inference	121
7.6	Version, Lifecycle, and Deprecation	122
7.7	Testing It from the UI	123
7.8	Interview Questions	125
7.9	Answers	126
8	AI Gateway, Model Registry & Routing Infrastructure	132
8.1	Overview	132
8.2	The Pipeline and Why Its Order Is Load-Bearing	133
8.3	The Registries	134
8.4	Rate Limiting: Reserve, Then Settle	135
8.5	Three Caches, Three Correctness Stories	136
8.6	Experiments: Canary and A/B Over Routing	138
8.7	Build or Buy	139
8.8	Testing It from the UI	139
8.9	Interview Questions	141
8.10	Answers	142
Part V Applied Multi-Model Systems		148
9	Agentic AI Model Routing	149
9.1	Overview	149
9.2	The Arithmetic of Roles	149
9.3	RoleSpec: Requirements, Not Model Names	150
9.4	Handoffs Must Carry Constraints	152
9.5	Resolving a Role, Once	153
9.6	Loop Prevention: Budget and Shape	153
9.7	The Supervisor and the Run Ceiling	154
9.8	Dynamic Agent Selection	155
9.9	Testing It from the UI	155
9.10	Interview Questions	157
9.11	Answers	158
10	Routing Across RAG, Tools, MCP & Multimodal Systems	164
10.1	Overview	164
10.2	Query Shape Decides Retrieval Strategy	164

10.3	Retriever Routing: Channels, Not a Retriever	166
10.4	Tool Routing: Permission Before Retrieval	168
10.5	Modality Routing: A Pipeline, Not a Model	170
10.6	Testing It from the UI	171
10.7	Interview Questions	172
10.8	Answers	173
11	Model Ensembles, Judges & Collaborative Intelligence	179
11.1	Overview	179
11.2	The Gate	179
11.3	Voting Is Only As Good As Its Equality Test	180
11.4	The Correlation Term	181
11.5	LLM-as-a-Judge and Its Three Biases	183
11.6	Mixture-of-Agents	184
11.7	Testing It from the UI	185
11.8	Interview Questions	187
11.9	Answers	188
Part VI	Operating the System	193
12	Inference Engineering for Multi-Model Systems	194
12.1	Overview	194
12.2	What a Replica Can Report	194
12.3	Routing on Engine State	196
12.4	The Continuous-Batching Curve	197
12.5	Capacity Planning	199
12.6	Quantisation as a Routing Input	200
12.7	Testing It from the UI	200
12.8	Interview Questions	202
12.9	Answers	203
13	Evaluation, Observability & Routing Intelligence	209
13.1	Overview	209
13.2	Building the Oracle Set	209
13.3	Scoring a Router Against It	210
13.4	The Confusion Matrix Is the Diagnosis	212
13.5	Observability: Four Signals With No Analogue	213
13.6	Drift Detection	214
13.7	Online Evaluation: Interleaving	214
13.8	Testing It from the UI	215
13.9	Interview Questions	217
13.10	Answers	218
14	Security, Governance & Enterprise Model Routing	224
14.1	Overview	224
14.2	Classify the Payload, Not the Caller	224
14.3	ABAC, Deny-First	227
14.4	Provider Trust and Retention	228

14.5 Prompt Injection Is an Authorisation Problem 229

14.6 Audit 230

14.7 Model Risk Governance 231

14.8 Testing It from the UI 232

14.9 Interview Questions 234

14.10 Answers 235

15 Adaptive Routing, Bandits, RL & Production System Design 241

15.1 Overview 241

15.2 Reward Design 241

15.3 The Bandit Ladder 243

15.4 Contextual: The Only Kind That Matches Routing 245

15.5 Exploration Under a Cost Ceiling 246

15.6 Mixture-of-Experts: The Same Problem Inside the Model 247

15.7 Testing It from the UI 248

15.8 The End-to-End Architecture 250

15.9 Interview Questions 251

15.10 Answers 252

Glossary 259

Closing Notes 263

References 264

PART I

Foundations

Before a system can route, it has to be clear about what it is choosing between and on what evidence. This part establishes the vocabulary that the rest of the book depends on: what separates routing from selection, why specialisation, cascades, ensembles, and fallbacks are four different patterns rather than four words for one, and how a model becomes a registry entry with measured cost, latency, quality, and risk attached. Get the registry wrong and every later decision inherits the error.

Model Routing Fundamentals

Subscribe → aiengineeringinsider.substack.com/subscribe

One model cannot serve a mixed workload economically, and the arithmetic that proves it also tells you what to build instead.

1.1 Overview

A production AI system receives a distribution of requests, not a request. In a typical assistant workload, somewhere between a third and a half of traffic is trivial — greetings, acknowledgements, single-field extractions, one-line classifications — and a thin tail of a few percent is genuinely hard: multi-hop analysis, non-trivial code, planning under constraints. The distribution is heavily skewed, and the cost of serving it on one model is set by the tail while the volume is set by the head.

Model routing is the practice of choosing, per request, which model serves it, from a catalog of deployments with different capabilities, prices, latencies, and trust properties. That sentence contains four separate design problems, and this book is organised around them: knowing what is in the catalog (Chapter 2), deciding what kind of request this is (Chapter 3), deciding how hard this instance is (Chapter 4), and deciding what to do when the first answer is wrong or absent (Chapter 5).

This chapter builds the vocabulary. It defines routing against the terms it is routinely confused with — model selection, cascades, ensembles, escalation, fallback, Mixture-of-Experts — shows why those distinctions are operational rather than pedantic, and presents the six-stage reference architecture that every later chapter refines. It ends with the design principles that the rest of the book applies without restating.

Model routing

A per-request decision function $r : (\text{request}, \text{context}, \text{catalog state}) \rightarrow \text{deployment}$ that maximises an objective over quality, cost, latency, and reliability, subject to hard constraints on capability, context length, data classification, and residency.

Three properties distinguish it from configuration. It is evaluated per request rather than per deployment. It reads live state — provider health, quota headroom, budget consumed — not only static profiles. And its output is explainable: a decision that cannot be replayed and justified is a decision you cannot operate.

1.2 Why Single-Model Architectures Fail

Single-model architectures do not fail loudly. They fail as a slow accumulation of four separate problems, each of which is individually survivable.

1.2.1 The cost argument

Consider a modest workload of one million requests per month with the mix in Table 1.1, and two candidate policies: everything on a capable general model at \$2.50/\$10.00 per million input/output tokens, or each class on the cheapest deployment that clears its quality floor.

Table 1.1: A representative assistant workload. Cost assumes 700 input and 400 output tokens per request; the single-model column prices every class at the large model’s rate. Figures come from the synthetic catalog shipped with the companion project.

Class	Share	Cheapest model that clears the floor	Routed \$/mo	Single \$/mo
Chitchat	22%	small hosted	\$76	\$1,265
Classification	14%	small hosted	\$48	\$805
Extraction	18%	small hosted	\$62	\$1,035
Summarisation	16%	small hosted	\$55	\$920
RAG QA	19%	open-weight 70B	\$77	\$1,093
Code	7%	coder specialist	\$27	\$403
Planning/math	4%	large (unchanged)	\$230	\$230
Total	100%		\$575	\$5,750

The ratio is ten to one, and its shape is more instructive than its size. Nothing was optimised about the hard cases: the planning row is identical in both columns, because those requests genuinely need the large model. The entire saving is in the other 96% of traffic, whose cost fell from \$5,520 to \$345. And notice what that does to the composition of the bill: after routing, 4% of requests account for 40% of spend, which is a much healthier place to be, because it means the money is now being spent on the requests that are actually difficult.

★ Key Insight

Routing economics are dominated by the head of the distribution, not the tail. Teams instinctively optimise the expensive requests because those are the ones they notice on a per-request basis. The money is in the cheap ones, because there are so many of them, and the risk of routing them wrong is low, which makes them the safest place to start.

1.2.2 The capability argument

Cost is the argument that gets attention; capability is the one that forces the architecture. No single model is simultaneously the best available at long-context retrieval, at code generation, at multilingual translation, at vision, and at cheap high-volume classification, and no single model is approved for every data class in a regulated enterprise. A model with a 128k context cannot serve a 300k-token document. A model without tool calling cannot serve an agent step. A model deployed only in `us-east` cannot serve a request whose payload must remain in the EU. These are not preferences that a weighted score can trade away; they are gates, and the distinction between gates and preferences is the single most important structural decision in a router (Section 1.7.1).

1.2.3 The availability argument

Every hosted model provider has rate limits, capacity incidents, and deprecation schedules. A single-model architecture inherits its provider’s availability exactly, with no recourse: when the provider returns 429s, the product is down. Multi-provider routing converts a hard dependency into a soft one, and that is usually the argument that persuades a review board even when the cost argument does not.

1.2.4 The evolution argument

Model releases arrive faster than integration cycles. A single-model architecture makes adopting a new model a migration project: prompts are tuned to one model’s quirks, evaluation is entangled with it, and the switch is all-or-nothing. A routing architecture makes it a catalog entry and a traffic percentage, which is the difference between a quarter of work and an afternoon.

! Pitfall / Warning

The failure mode of a single-model system is not an outage; it is a slow drift into a position where every improvement is expensive. By the time cost is visible enough to trigger a project, the prompts, evaluations, and guardrails are all coupled to one model. Introduce the routing seam before you need it, even if version one routes everything to the same place.

1.3 Model Selection Versus Model Routing

These two terms are used interchangeably in job descriptions and mean very different things in an architecture review.

Table 1.2: Selection is a design-time activity; routing is a runtime system.

Dimension	Model selection	Model routing
When	Design time, per project or per feature	Runtime, per request
Input	Benchmarks, an evaluation set, a procurement negotiation	The request, its classification, live catalog and provider state
Output	A default and an evaluation report	A deployment, a fallback chain, and an explanation
Changes	On a review cycle	On every request, and on every catalog or health change
Failure mode	The chosen model drifts out of date	The decision function misroutes, silently
Owned by	An engineer with an eval set	A service with an SLO
Instrumentation	An offline report	Distribution, escalation rate, cost regret, decision latency

Both are needed. Selection produces the quality vector that the router scores against; without it, routing is a confident guess. Routing turns that vector into per-request behaviour; without it, selection is a constant in a config file. The common failure is doing selection well, writing the result into `DEFAULT_MODEL`, and calling the system multi-model because it can be changed by editing an environment variable.

1.4 Multi-Model Systems and the Five Composition Patterns

A **multi-model system** uses more than one model in production. That is a weak statement on its own, because there are at least five structurally different ways to do it, with different cost curves, different failure modes, and different things to monitor. Interviewers probe this distinction because candidates routinely describe a cascade and call it an ensemble.

Table 1.3: Five composition patterns. Note the trigger column: it is what actually separates them in an implementation.

Pattern	Models per request	Trigger and purpose	Watch
Specialisation	Exactly one	The request's class selects a model built or tuned for it. Purpose: quality per dollar.	Misclassification rate
Cascade	One, sometimes more	A cheap model answers; a verifier judges; failure escalates upward. Purpose: pay for capability only when needed.	Escalation rate, verifier precision
Ensemble	Always n	n models answer in parallel and an aggregator picks or merges. Purpose: variance reduction or error decorrelation.	Agreement rate, cost multiple
Escalation	One, sometimes more	A <i>quality</i> verdict moves the request up a capability ladder. (The mechanism inside a cascade.)	Escalation rate by class
Fallback	One, sometimes more	An <i>availability</i> failure moves the request sideways to a peer. Purpose: survive a provider.	Fallback rate, provider diversity

1.4.1 Specialisation

Specialisation routes by what the request *is*. A code-tuned 32B model can beat a general frontier model on code generation at a tenth of the price, because fine-tuning concentrated its capacity on the distribution you care about. The catch is coverage: a specialist is worse outside its domain, often sharply, so specialisation depends entirely on classification quality. A router that sends 8% of general questions to a coder model has traded a cost saving for a quality cliff on 8% of traffic, and the cliff will not appear in an average.

1.4.2 Cascades

A cascade runs the cheap model first, checks the answer, and escalates only on failure. Its economics are governed by one equation. With small-model cost c_s , large-model cost c_l , verifier cost c_v , and small-model sufficiency rate p :

$$\mathbb{E}[\text{cost}] = c_s + c_v + (1 - p)c_l$$

The cascade beats always-large when $c_s + c_v + (1 - p)c_l < c_l$, that is when

$$p > \frac{c_s + c_v}{c_l}.$$

With a small model at 6% of the large model’s price and a verifier at 2%, the cascade wins as soon as the small model suffices more than 8% of the time. That threshold is astonishingly low, which is why cascades are the default recommendation—and why the interesting question is never “should we cascade” but “what is our verifier, and what is its false-accept rate”. A verifier that wrongly accepts 10% of bad answers has converted a quality problem into an invisible one.

1.4.3 Ensembles

An ensemble multiplies cost by n and must buy something specific in return: variance reduction, error decorrelation across genuinely different models, or a calibrated confidence signal from the agreement rate. If none applies, it is an n -times bill for a rounding error. Chapter 11 gives the gate that decides.

1.4.4 Escalation and fallback

These are the two that get conflated, and conflating them is expensive. Escalation is triggered by a *verdict*: the model answered, and the answer was judged insufficient. Fallback is triggered by an *exception*: the model did not answer. Escalation moves *up* a capability ladder and costs more by design. Fallback moves *sideways* to a peer of comparable capability, ideally at a different provider, and should cost roughly the same.

A system that escalates on a 429 will spend frontier-model money every time a provider has a capacity blip—and it will do so precisely during the incident, when spend is least observable. A system that treats a bad answer as a failover will retry the same class of model at a different vendor and produce the same bad answer. Chapter 5 builds both correctly.

From the Field

The clearest smell of the two being conflated is a metric called *retry rate*. If your dashboard has one number for retries, you cannot tell a quality regression from a provider incident, and the two get debugged by different people. Split it on day one: `routing.escalation` and `routing.attempt_failed`, with the error kind as a label on the second.

1.5 Model Routing Versus Mixture-of-Experts

This comparison is a standard interview probe, because the words are similar and the systems are not.

Table 1.4: Two routers at different levels of the stack.

Dimension	System-level model routing	Mixture-of-Experts
Granularity	One decision per request	One decision per token, per layer
Unit routed to	A deployment: weights, hardware, vendor, region	A feed-forward expert inside one forward pass
Learned how	Rules, classifiers, or bandits over production feedback	Jointly with the model, by gradient descent
Latency budget	Sub-millisecond to a few milliseconds	Nanoseconds; it is inside the kernel
Failure mode	Misroute: wrong model, wrong price, wrong jurisdiction	Expert collapse and dropped tokens under capacity limits
Balancing	Explicit traffic shaping and exploration	An auxiliary load-balancing loss
Observable	Yes: every decision is a log line	Only through aggregate expert-utilisation statistics

The useful part of the comparison is what the two share. Both need a gate that is cheap relative to what it gates. Both suffer load imbalance when the gate is pure argmax, because the winner accumulates all the traffic and the losers’ statistics never improve. And both fix it the same way: MoE adds an auxiliary balancing loss, a production router adds bounded exploration (Chapter 15). If you can state that parallel, you have understood both.

1.6 Model Routing in Agentic AI

In an agent, routing stops being one decision and becomes many. A single agent run is a sequence of roles—plan, retrieve, execute, criticise, summarise—each with different requirements, and the call counts are wildly uneven: the planner runs once, the executor runs on every tool turn. Routing per role rather than per agent is therefore the largest single cost lever in agent systems, because the role that dominates the call count also dominates the bill. Chapter 9 develops this, including the two failure modes that only appear here: handoffs that lose the caller’s constraints, and routing loops between agents that a hop budget alone does not stop.

1.7 Core Architecture of a Model Router

Figure 1.1 is the reference architecture this book builds. Every later chapter deepens one of its six stages.

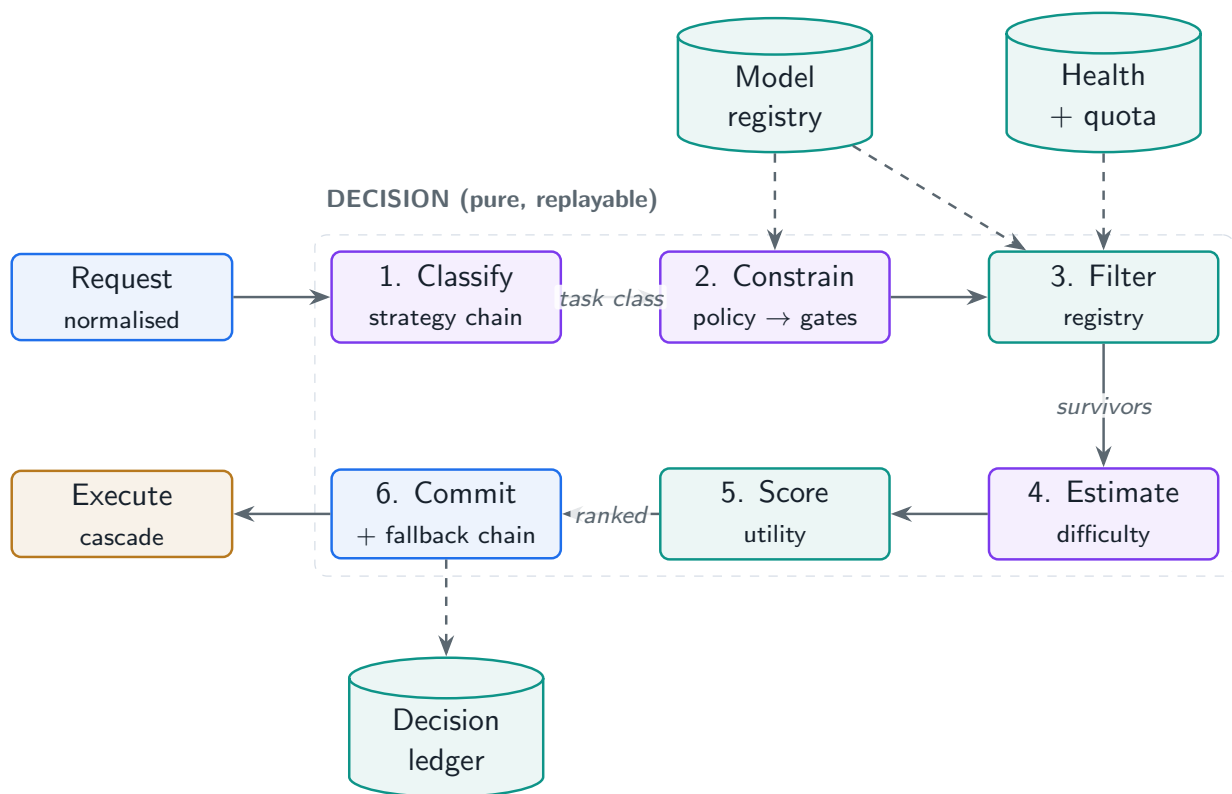


Figure 1.1: The six-stage reference router. Filtering strictly precedes scoring; execution is a separate component from the decision, which is what makes decisions replayable offline against production logs.

1. **Classify** What kind of work is this? A chain of strategies ordered by cost, stopping at the first one confident enough (Chapter 3).
2. **Constrain** Turn policy and classification into *hard gates*: required capabilities, minimum context, permitted regions, data risk tier (Chapters 3 and 14).
3. **Filter** Ask the registry which deployments survive the gates, and keep a reason for every one that did not (Chapter 2).
4. **Estimate** How hard is *this instance*? Difficulty narrows the candidate set to a tier band (Chapter 4).
5. **Score** Rank the survivors on normalised quality, cost, latency, and reliability under a weight vector that depends on user tier and remaining budget (Chapter 6).
6. **Commit** Take the winner, and build a fallback chain that prefers provider diversity over utility rank (Chapter 5).

Two structural choices in that diagram are worth defending explicitly, because they are the ones candidates most often get wrong.

1.7.1 Filtering strictly precedes scoring

It is tempting to express everything as a score: give tool-calling support a weight, give EU residency a weight, and let the optimiser sort it out. This is wrong, and it is wrong in a way that produces incidents rather than inefficiency. A weighted score means a sufficiently cheap model can outvote a constraint. The cheapest model in your catalog will, eventually, win a request it

cannot physically serve—or worse, one it is not *permitted* to serve, and the request will have left your network before anyone notices.

Constraints are binary. Preferences are weighted. Keeping them in separate stages is what makes the second stage safe.

1.7.2 Deciding and executing are separate components

The decision stage is pure: given a request and a snapshot of catalog and health state, it returns a decision. Nothing in it performs I/O against a model provider. That purity is not aesthetic. It means you can replay six months of production requests through a modified router in seconds and compare decisions, which is how routing regressions are actually caught (Chapter 13). A router that can only be evaluated by putting it in front of live traffic will be changed rarely and nervously.

1.8 Model Routing Design Principles

These ten principles are applied throughout the book without restatement.

1. **Filter, then score.** Capability, context, residency, and data class are gates, never weights.
2. **Optimise cost per successful task.** Dividing by the acceptance rate is the only form of the metric that cannot be gamed by a cheap model that fails often.
3. **Keep the router cheaper than the route.** Order strategies by cost and stop at the first confident one; classify on a truncated prompt.
4. **Separate quality escalation from availability failover.** Different triggers, different directions, different metrics, different alerts.
5. **Classify the payload before choosing the vendor.** Egress is irreversible.
6. **Calibrate every confidence you threshold on.** Raw model and classifier confidences are systematically overconfident.
7. **Prefer provider diversity in fallback chains.** Correlated fallbacks are decoration.
8. **Persist the candidate vector, not the winner.** The scalar score cannot answer “why that one” three months later.
9. **Evaluate against the cheapest acceptable model.** Comparing against the best model measures your catalog, not your router.
10. **Explore on purpose, with a bounded budget.** A pure-argmax policy cannot discover that a new model is better.

✓ Best Practice

Ship the seam before the intelligence. Version one of a router should have the full six-stage shape with a rule chain that sends everything to one model. It costs a day, it produces the decision ledger immediately, and the ledger is what tells you which of the later chapters you actually need.

1.9 Production Code: The Decision Object

Everything above reduces to two types. The request is normalised so that routing never depends on a provider's wire format; the decision carries not just the winner but the whole candidate set, the rejection reasons, and the fallback chain.

```
app/routing/schemas.py
```

```
class RoutingRequest(BaseModel):
    """Normalised request. The gateway never routes on a raw provider payload."""
    prompt: str
    task_class: TaskClass | None = None # None -> classify
    tenant_id: str = "anonymous"
    user_tier: Literal["free", "pro", "enterprise", "internal"] = "free"
    conversation_id: str | None = None
    estimated_output_tokens: int = 512
    attachments: list[Modality] = Field(default_factory=list)
    constraints: RoutingConstraints = Field(default_factory=RoutingConstraints)
    weights: dict[str, float] | None = None # objective weights
    metadata: dict[str, Any] = Field(default_factory=dict)

class RouteCandidate(BaseModel):
    """A model that survived filtering, with its scored components exposed.

    Exposing the components is not a debugging nicety. Every routing incident
    review starts with "why did it pick that?", and a scalar score cannot
    answer it. Persisting the vector makes the question answerable months
    later without a repro.
    """
    model_id: str
    utility: float = 0.0
    quality: float = 0.0
    est_cost_usd: float = 0.0
    est_latency_ms: float = 0.0
    reliability: float = 1.0
    components: dict[str, float] = Field(default_factory=dict)
    penalties: dict[str, float] = Field(default_factory=dict)

class RoutingDecision(BaseModel):
    """What the router chose, why, and what it would have chosen instead."""
    model_id: str
    strategy: str # which strategy fired
    task_class: TaskClass
    confidence: float = 1.0
    reason: str = ""
    candidates: list[RouteCandidate] = Field(default_factory=list)
    rejected: dict[str, str] = Field(default_factory=dict) # model -> why
    fallback_chain: list[str] = Field(default_factory=list)
    escalation_ladder: list[str] = Field(default_factory=list)
    escalation_allowed: bool = True
    decision_ms: float = 0.0
```

```
trace_id: str | None = None
```

The `rejected` map is the field teams add last and wish they had added first. When a router returns nothing, or returns something surprising, the only question is which gate removed the model you expected, and a map from model id to a human-readable reason answers it without a debugger.

The engine's public method is the six stages in order, and it is worth reading as a specification rather than as code.

```
app/routing/engine.py
```

```
async def route(self, req: RoutingRequest) -> RoutingDecision:
    started = time.perf_counter()

    # 1. classify -- cheapest strategy that is confident enough wins
    signal, strategy_trace = await self.chain.resolve(req)
    task = signal.task_class or TaskClass.CHITCHAT

    # 2. constrain -- capabilities discovered while classifying are now gates
    req.constraints.required_capabilities |= signal.extra_capabilities
    req.constraints.min_context_window = max(
        req.constraints.min_context_window,
        estimate_tokens(req.prompt) + req.estimated_output_tokens + 512)

    # 3. filter -- hard gates only, with a reason for every rejection
    survivors, rejected = self.reg.eligible(req, task)
    gate_survivors = list(survivors)      # ladder is built from these

    # 4. estimate -- instance difficulty narrows the tier band
    difficulty, difficulty_trace = await self.complexity.estimate(req, task)
    survivors, tier_rejects = self._apply_band(survivors, difficulty, signal)
    rejected.update(tier_rejects)

    if not survivors:
        # Never silently downgrade a constraint. This is a 503 with a
        # machine-readable body, because the first question in the incident
        # is always which gate emptied the set.
        raise NoEligibleModel(task, rejected)

    # 5. score -- normalised multi-objective utility
    candidates = self._score(req, task, survivors, difficulty)

    # 6. commit -- winner plus a provider-diverse fallback chain
    chosen = candidates[0]
    return RoutingDecision(
        model_id=chosen.model_id, strategy=signal.strategy, task_class=task,
        candidates=candidates, rejected=rejected,
        fallback_chain=self._fallback_chain(chosen, candidates),
        escalation_ladder=self._ladder(task, gate_survivors),
        escalation_allowed=difficulty.band != "trivial",
        decision_ms=round((time.perf_counter() - started) * 1000, 3))
```


1.10 Testing It from the UI

Lab 1 runs a six-request workload through the router and through a single-model baseline, and shows both bills side by side.

1. Start the stack: `make dev` (or `uvicorn app.main:app --reload` and `npm run dev` in `frontend/`).
2. Open <http://localhost:3000/labs/1>.
3. Press **Run**. The table shows, per request, the routed model, the routed cost, and what the same request would have cost on the baseline.
4. Change **Baseline model** to the reasoning deployment and rerun. The saving grows; the quality delta does not.
5. Turn **Enforce tier bands** off and rerun. Watch cheap models start winning planning requests, and watch the mean quality fall by more than the cost falls.

```
lab 1 -- routed vs single-model, default workload
```

routed_cost_usd	0.008922			
single_cost_usd	0.031000			
saving_pct	71.22			
routed_mean_quality	0.8267			
single_mean_quality	0.8850			
quality_delta	-0.0583			
chitchat	-> vendor_a/small@us	\$0.000308	vs \$0.005137	q 0.80 / 0.89
extraction	-> vendor_a/small@us	\$0.000310	vs \$0.005168	q 0.89 / 0.93
summarisation	-> cluster_eu/mistral-24b	\$0.000058	vs \$0.005155	q 0.81 / 0.92
code_generation	-> vendor_c/coder-32b	\$0.000236	vs \$0.005163	q 0.87 / 0.88
math_reasoning	-> vendor_c/coder-32b	\$0.000238	vs \$0.005180	q 0.70 / 0.82
planning	-> vendor_b/large@us	\$0.007773	vs \$0.005197	q 0.89 / 0.87

Read that output the way a design review would. The headline is a 71% saving, and the headline is not the finding. There are three findings.

Planning cost more than the baseline (\$0.0078 against \$0.0052) and bought two quality points. The difficulty estimator put that request in the hard band, the band restricted the candidate set to the top tiers, and the scorer picked the strongest model available. That is the router working correctly: routing is not a cost-reduction function, it is an allocation function, and correct allocation sometimes spends more.

Maths still lost twelve quality points. The arithmetic prompt was classified as code generation by a rule that fires on digits and operators, so it went to the coder specialist. That is a classification error with a quality consequence, and it is visible here only because the table is per-class.

Summarisation moved to the self-hosted cluster at a fifth of a cent per thousand requests, losing eleven quality points. Whether that trade is correct depends entirely on a per-class quality floor (Chapter 6) — which is why the default configuration in the repository has one, and why turning it off is the second thing the lab lets you do.

A mean quality delta of -0.058 summarises none of that; it averages a deliberate upgrade, a misclassification, and a policy trade into one number that supports no decision.

! Pitfall / Warning

Never accept a routing change on aggregate metrics. Cost is additive across classes, so it aggregates honestly; quality is not, because the classes where quality matters most are usually the smallest by volume. Always evaluate per class, and always look at the smallest class first.

1.11 Interview Questions

Exercise 1.1 – System design

Design the routing layer for a customer-support assistant handling 40 million requests per month across three regions, with a p95 latency SLO of 3 seconds, a mixed workload from greetings to contract analysis, and a requirement that EU customer data never leaves the EU. Walk through the components, the decision order, and what you would measure in week one.

Hint. Start from the six stages. Decide where residency enters, and be explicit about which stages are hard filters and which are soft.

Exercise 1.2 – Production

Your router has been stable for two months. On Tuesday, traffic to the large model rises from 12% to 31% with no deploy, and spend rises accordingly. Answer quality is unchanged. Give a prioritised list of hypotheses and the query or dashboard that discriminates between them.

Hint. Nothing in the router changed. Something upstream of it did. What are the router's inputs?

Exercise 1.3 – Debugging

A customer reports that a request with an attached PDF returned “I cannot read attachments”. Your logs show the router selected a text-only model, and the model was the cheapest candidate. Where is the bug, and what class of bug is it?

Hint. Which stage should have removed that model, and what would have had to be true for it to survive?

Exercise 1.4 – Technical

Derive the break-even condition for a two-stage cascade, then state what happens to it when the verifier has a false-accept rate f and a false-reject rate g . Which of the two errors is more expensive, and why?

Hint. Write the expected cost with the verifier's errors included, then ask what each error costs: one is money, the other is something you cannot see.

Exercise 1.5 – Case study

A team replaced their single frontier model with a router and reported a 71% cost reduction with “no measurable quality change”. Six weeks later, enterprise renewals dropped and support escalations rose 40%. Reconstruct what most likely happened, and describe the evaluation design that would have caught it before launch.

Hint. “No measurable quality change” is a claim about an aggregate and a sample size. Which segment of traffic is both small and disproportionately valuable?

1.12 Answers**Answer 1.1 – System design**

Design the routing layer for a 40M-request/month multi-region support assistant with a 3 s p95 SLO and an EU residency requirement.

Shape of the answer. 40M/month is about 15 requests per second average and, at a typical 2.5:1 peak-to-mean ratio, roughly 40 rps at peak. That is small enough that the router itself is not a scaling problem and large enough that a 10 ms routing overhead is 400 core-milliseconds per second of pure tax. State that arithmetic first; it sets the budgets for everything else.

Components, in decision order.

Stage 0, normalisation and classification of the payload. Before any routing, run PII and residency detection on the request. This is the first stage, not a later one, because the output is a hard gate and because egress is irreversible. For an EU tenant, the detector’s result and the tenant’s declared region jointly produce `allowed_regions = ["eu-west"]`.

Stage 1, classify. A three-strategy chain: metadata (the support console already knows the ticket category—use it, do not re-infer it), rules for the obvious verbs, and a semantic centroid classifier for the rest. No LLM router in version one: at 40 rps the added latency and cost are real and the residual it would fix is unknown until the ledger exists.

Stage 2–3, constrain and filter. Gates are: required capability (tool calling for the “take an action” intents, structured output for the extraction intents), minimum context (ticket threads with attachments run long, so compute it from the actual prompt), region, and risk tier. The registry returns survivors and a rejection map.

Stage 4, estimate. Static features only at first: they cost microseconds. Difficulty narrows to a tier band.

Stage 5, score. Weights by user tier. Crucially, the SLO enters here as a *penalty on estimated p95 latency*, and a hard rejection when the estimate exceeds the remaining deadline. With a 3 s p95 budget and roughly 400 output tokens, that already excludes any deployment whose p95 TPOT is above about 6 ms per token once you reserve time for retrieval and guardrails—which in practice means reasoning models are out of scope for the interactive path entirely and must be moved to an asynchronous queue.

Stage 6, commit. Fallback chain prefers a different provider in the same region. For EU traffic this is the constraint that hurts: many providers have fewer EU deployments, so the honest answer is that EU tenants get a shorter fallback chain, and the mitigation is a self-hosted EU cluster as the final rung.

Regional topology. Route within region, not across. A global router that can select a US deployment for an EU request is one config error away from a notifiable incident. Run one gateway per region with a region-scoped catalog; share the catalog *definition* through configuration, not the runtime.

Week-one measurements. Decision distribution by class and region; p50/p95/p99 routing overhead; escalation rate and fallback rate as separate series; cost per successful task by class; and the rejection map's top reasons, which tell you whether your gates are doing what you think. Then, and only then, decide whether Chapter 4's machinery is warranted.

Answer 1.2 – Production

Traffic to the large model rises from 12% to 31% with no deploy and no quality change. Diagnose.

The router is a function of its inputs, and none of the code changed, so one of the inputs did. In rough order of likelihood:

1. *The prompt template changed upstream.* Someone added a system preamble, a few-shot block, or a retrieval step that prepends more context. That raises the estimated input token count, which raises the computed `min_context_window`, which silently filters out short-context deployments. This is the single most common cause and it is invisible in a diff of the router. **Discriminator:** plot mean prompt tokens by class over the same window; look for a step change aligned with the shift.
2. *The traffic mix changed.* A new integration, a marketing campaign, or a seasonal pattern shifted the class distribution towards harder classes. **Discriminator:** decision distribution *conditioned on task class*. If the within-class distributions are unchanged and only the class mix moved, the router is behaving correctly and the answer is a capacity and budget conversation, not a bug.
3. *A cheaper deployment is failing its health check or is rate-limited.* If the small model's breaker is opening intermittently, or its provider quota is exhausted mid-day, the survivors set loses its cheap members and the large model wins by default. **Discriminator:** breaker state transitions and `routing.attempt_failed` by model; also check for a daily sawtooth in the shift, which is the signature of a quota that resets.
4. *A catalog change.* Someone edited quality scores, prices, or tags after an evaluation run. The catalog is data, so it changes without a deploy — which is a feature until it is an incident. **Discriminator:** catalog version history; the registry exposes a monotonic version, and the decision ledger should record which version made each decision.
5. *Calibration drift in the difficulty estimator.* If difficulty is estimated by a kNN over labelled traffic and the traffic drifted, more requests land in the “hard” band. **Discriminator:** distribution of difficulty scores over time, not just the band assignment.

The fact that quality is unchanged is itself evidence: it argues against a model regression and for a mechanical cause. Report the population stability index of the decision distribution against the pre-shift baseline (Chapter 13); above 0.25 this should have paged before anyone noticed the invoice.

Answer 1.3 – Debugging

A PDF request went to a text-only model, which was the cheapest candidate. Where is the bug?

The bug is architectural, not numerical: a *constraint was expressed as a preference*. Somewhere, “supports document input” contributes to a score rather than removing ineligible models from the candidate set. With enough price advantage, a text-only model outvotes it. This is exactly the failure mode Section 1.7.1 exists to prevent.

There are three plausible mechanisms, and they are worth distinguishing because they have different fixes:

(a) *The attachment never reached the router.* If the API accepted the PDF but the normalisation layer did not populate `req.attachments`, the router had no signal to gate on. The gate worked; its input was empty. Fix: assert in normalisation that every attachment is represented, and add a test that a request with a document body but empty `attachments` is rejected outright rather than routed.

(b) *The modality was scored.* Fix: move it into `RoutingConstraints.required_modalities` and let the registry’s `eligible()` remove non-matching models with the reason “no input modality document”.

(c) *The gate emptied the set and the code fell back.* If no vision deployment was healthy, a defensive `or survivors` clause may have restored the full set. Soft rules (quality floors, tier bands) may do that; hard gates never may. Fix: split the two code paths and let hard-gate starvation raise `NoEligibleModel`, which becomes a 503 with the rejection map—a clean failure the caller can act on, rather than a confidently wrong answer.

The general lesson for the interview: any bug of the form “the cheap model was chosen for something it cannot do” is a filter-versus-score bug until proven otherwise, and the diagnostic artefact is the rejection map. If the map does not contain the vision models with a modality reason, the gate never ran.

Answer 1.4 – Technical

Derive the cascade break-even, then add verifier error rates.

Ideal verifier. With small cost c_s , large cost c_l , verifier cost c_v , and small-model sufficiency p :

$$\mathbb{E}[\text{cost}] = c_s + c_v + (1 - p)c_l < c_l \iff p > \frac{c_s + c_v}{c_l}.$$

With $c_s = 0.06 c_l$ and $c_v = 0.02 c_l$, break-even is $p > 0.08$.

Imperfect verifier. Let f be the false-accept rate (a bad answer passes) and g the false-reject rate (a good answer is escalated). The probability of escalating becomes

$$P(\text{escalate}) = (1 - p)(1 - f) + pg,$$

so

$$\mathbb{E}[\text{cost}] = c_s + c_v + [(1 - p)(1 - f) + pg]c_l.$$

The break-even condition tightens to

$$p(1 - g) + (1 - p)f > \frac{c_s + c_v}{c_l},$$

and note what that says: false accepts *help* the cost equation. That is the trap.

Which error is more expensive. False rejects cost money and are visible: they show up as an elevated escalation rate and a bigger bill, and someone will investigate. False accepts cost quality and are invisible: the answer is served, the user may or may not complain, and

the cascade reports excellent economics precisely *because* it is shipping bad answers cheaply. A cascade with a lazy verifier looks better on every dashboard you have.

The engineering consequences follow directly. Tune the verifier for high recall on failure (accept a higher g) rather than for cost. Instrument the false-accept rate directly by sampling accepted small-model answers and grading them with a stronger model or a human, on a schedule, forever — it is the one number in a cascade that no other metric will surface. And treat a falling escalation rate with suspicion: the pleasant explanation is that the small model improved; the likely one is that the verifier got looser.

Answer 1.5 – Case study

71% cost reduction, “no measurable quality change”, then enterprise renewals fall and escalations rise 40%.

What happened. The evaluation measured a mean over a sample dominated by the head of the distribution. Roughly 90% of requests are easy, so a sample drawn proportionally is roughly 90% easy requests, and moving those to a small model genuinely does not change their quality. The remaining few percent — multi-turn contract analysis, complex troubleshooting, anything an enterprise account manager actually cares about — were also moved down, lost real quality, and were too rare in the sample to move the mean beyond its noise band. “No measurable change” was true and irrelevant: the study was underpowered exactly where it mattered.

Three compounding factors are typical:

Value is not uniform across requests. Enterprise accounts generate a small share of requests and a large share of revenue. Traffic-weighted evaluation systematically underweights them. *Degradation was gradual and attributable elsewhere.* Users do not file tickets saying “the model got smaller”. They ask again, rephrase, escalate to a human, and eventually churn. The 40% escalation rise is the leading indicator; renewals are the lagging one, six weeks later.

No per-class floor. With only an aggregate objective, nothing stopped the optimiser from taking quality out of the classes where it was cheapest to take, which are the hard ones.

The evaluation that would have caught it.

Stratify, then weight. Sample per task class with a minimum of a few hundred per class regardless of natural frequency, then reweight for reporting. Report per-class deltas with confidence intervals and refuse to report a single aggregate quality number.

Segment by account value and by conversation depth. Turn 6 of a conversation is not turn 1; long conversations are where context handling and consistency break, and they are disproportionately enterprise.

Set acceptance thresholds per class before the experiment. Define what a regression is in advance, per class, and let any class breach block the rollout regardless of the aggregate.

Ramp with an automatic rollback on a leading indicator. Escalation-to-human rate, regeneration rate, and conversation length are all available within hours; renewals are not. Chapter 7’s migration plan holds at 1%, 5%, 25% with a hold period and rolls back on an acceptance delta rather than on an error rate.

Keep a permanent holdout. A small, fixed slice of traffic stays on the previous policy indefinitely. It is the only way to detect slow degradation once the change is fully rolled out and the counterfactual would otherwise be gone.

Chapter Summary

Model routing is a per-request decision function over a catalog, constrained by capability and governance and optimised over quality, cost, latency, and reliability. Single-model architectures fail on four independent axes— cost, capability, availability, evolution—and the cost failure is dominated by the head of the request distribution rather than its tail. The five composition patterns (specialisation, cascade, ensemble, escalation, fallback) differ in what triggers them and in what you must monitor; conflating escalation with fallback is the most expensive of the available confusions. System-level routing and Mixture-of-Experts share a gate-cost problem and a load-balancing problem at wildly different granularities. The reference architecture has six stages, and its two load-bearing structural choices are that filtering strictly precedes scoring, and that deciding is separate from executing so that decisions can be replayed offline.

LLM Capabilities, Model Profiling & Selection

Subscribe → aiengineeringinsider.substack.com/subscribe

A router is only as good as the catalog it reasons over, and most catalogs are a spreadsheet of marketing numbers.

2.1 Overview

The registry is the least glamorous component in a routing system and the one that determines whether any of the rest can work. Every decision the router makes is a query against it: which models can serve this modality, which are approved for this data class, which are cheap enough, which are fast enough, which are good enough. If those answers are wrong, a perfect decision engine produces confidently wrong decisions.

This chapter builds the catalog. It starts from the single most consequential modelling choice — that capabilities are *binary gates* and quality is a *measured vector*, and that the two must never be mixed — and then works through the model families you will actually route between, the seven properties every profile must carry, how to measure each one honestly, and how to build a registry that survives a provider deprecating a model on thirty days' notice.

Model profile

A record describing one *deployment* — a specific model, at a specific version, on a specific provider, in a specific region — carrying its capability set, context limits, price, measured latency distribution, per-task quality vector, reliability, rate limits, and the maximum data risk tier it is approved for.

The unit is the deployment, not the model family. The same weights on two providers, or in two regions, are two profiles, because they differ in the two fields that most often decide a route: residency and measured latency.

2.2 The Capability Matrix

A **capability matrix** is a table of models against binary capabilities. Its purpose is to answer one question — *can this model physically serve this request?* — and it must answer only that question.

Table 2.1: A capability matrix. Every cell is yes or no. Anything that would sensibly be a number belongs in the quality vector, not here.

Deployment	Tools	JSON	Vision	Think	Cache	Context
local/qwen2.5-1.5b	—	yes	—	—	—	32k
local/qwen2.5-7b	yes	yes	—	—	—	32k
vendor_a/small@us	yes	yes	—	—	yes	128k
vendor_a/large@us	yes	yes	yes	—	yes	200k
vendor_b/large@us	yes	yes	yes	—	yes	200k
vendor_b/reasoner@us	yes	yes	—	yes	yes	200k
vendor_c/coder-32b	yes	yes	—	—	—	131k
cluster_eu/mistral-24b	yes	yes	—	—	—	65k

★ Key Insight

The test for whether a property belongs in the capability matrix is: *can its absence cause the request to fail outright?* No tool calling means an agent step cannot run. No vision means an image request cannot be served. Those are gates. “Good at code” cannot cause a failure; it causes a worse answer, so it is a score. Mixing the two lets a price advantage outvote a physical impossibility.

In code, the vocabulary is closed and the profile carries a set:

```
app/routing/schemas.py
```

```
class Capability(str, Enum):
    """Binary capabilities. A model either has one or it does not.

    Binary is the point: graded capability ("good at coding") belongs in the
    quality vector, where it can be measured. Anything that can silently fail
    a request -- no tool calling, no JSON mode, no vision -- must be a hard
    filter, not a score, or the router will happily pick a model that cannot
    physically serve the request.
    """
    TOOL_CALLING = "tool_calling"
    PARALLEL_TOOL_CALLS = "parallel_tool_calls"
    STRUCTURED_OUTPUT = "structured_output"
    JSON_MODE = "json_mode"
    STREAMING = "streaming"
    EXTENDED_THINKING = "extended_thinking"
    PROMPT_CACHING = "prompt_caching"
    LOGPROBS = "logprobs"
    FINE_TUNED = "fine_tuned"
```

2.3 Model Families and What They Are For

The catalog you route over is not a list of models; it is a small number of *families* with different cost-quality curves. Knowing which family a request belongs to is most of routing.

Table 2.2: Model families as routing targets. The last column is the one that matters at design time.

Family	What it is	Route to it when
Small general	1–8B, cheap, fast, high throughput. Strong on classification and extraction, weak on multi-step reasoning.	The task is pattern-shaped: classify, extract, route, rewrite, greet.
Mid general	20–70B class, the workhorse tier. Most production traffic belongs here.	The task needs comprehension but not deliberation.
Large general	Frontier models. Best broad quality, highest price per token.	The task is open-ended, high-stakes, or long-context, and cheaper tiers measurably fail it.
Reasoning	Trained to spend test-time compute before answering. Dramatically better on maths, proofs, planning; TTFT measured in seconds.	The task has a verifiable answer that requires derivation <i>and</i> the latency budget is asynchronous.
Coding	Fine-tuned on code and diffs. Often beats a general model two tiers above it, at a tenth of the price.	The task is code generation, review, or transformation.
Embedding	Produces vectors, not text. Priced per input token only, latency in tens of milliseconds.	Retrieval, semantic routing, clustering, deduplication.
Vision	Accepts images and often documents.	There is an image or a scanned page in the request.
Audio	Speech-to-text or speech-to-speech.	There is audio, and the pipeline decision (Chapter 10) is not to transcribe first.
Multimodal	Accepts several modalities natively in one context.	Cross-modal reasoning is genuinely required, not just extraction from each modality.
Fine-tuned	Your weights, your distribution.	You have thousands of labelled examples and a stable task; see the warning below.

2.3.1 Reasoning models are a scheduling decision

Reasoning models change the shape of the latency profile, not just its magnitude. A model that generates thousands of internal tokens before its first visible output has a TTFT measured in seconds, and no amount of streaming hides it. That makes “should we use a reasoning model” a question about the *interaction pattern* first and quality second: if the user is waiting on a chat cursor, a three-second TTFT is a product decision that someone senior needs to make; if the work is a background job, it is free.

The corollary is a routing rule worth stating explicitly: a request with an interactive latency SLO should not have reasoning models in its candidate set at all. Filter them out by latency estimate rather than hoping the utility function will handle it.

2.3.2 Fine-tuned models: the routing cost nobody budgets

A fine-tuned model is a specialist you own, and it is genuinely the best answer for narrow, high-volume, stable tasks. The routing cost is that it adds a deployment whose quality vector is sharply peaked: excellent inside its distribution, unpredictable outside it. That makes classification accuracy, not model quality, the binding constraint. Before adding one, ask what fraction of misrouted traffic it will receive and what that traffic will experience.

2.3.3 Open-weight against proprietary

Table 2.3: The trade is not primarily about quality any more.

Dimension	Open-weight (self-hosted)	Proprietary API
Cost model	Fixed: GPU-hours. Cheap at high utilisation, terrible at low.	Variable: per token. Perfectly elastic, never zero.
Break-even	Roughly when sustained throughput fills a GPU for most of the day. Below that, the API wins.	—
Residency	Wherever you put it. The strongest argument.	Wherever the vendor offers a region.
Data posture	No third-party egress at all.	Contractual; verify retention and training terms per tier.
Latency	Yours to tune: batching, quantisation, hardware.	Yours to measure, not to change.
Deprecation	You control it. Weights do not disappear.	30–180 days' notice, on the vendor's schedule.
Operational load	Real: capacity, upgrades, incident response, GPU supply.	Near zero.
Quality ceiling	Below the frontier, above what most tasks need.	The frontier.

The mature architecture uses both and routes between them, which is why the catalog treats a self-hosted cluster as just another provider with a different price model. Converting GPU-hours into a per-token price is what makes them comparable, and the term teams forget is utilisation:

`app/routing/economics.py`

```
def gpu_amortised_price(gpu_usd_per_hour: float, tokens_per_second: float,
                       utilisation: float = 0.45) -> PricePoint:
    """Convert a self-hosted deployment into a per-token price.

    Utilisation is the term teams forget. A cluster that is busy 45% of the
    day costs the same as one busy 100% of the day, so the honest per-token
    price is roughly 2.2x the number a peak-throughput benchmark suggests.
    """
    util = min(max(utilisation, 0.01), 1.0)
    tokens_per_hour = tokens_per_second * 3600 * util
    per_mtok = gpu_usd_per_hour / max(tokens_per_hour, 1.0) * 1_000_000
    return PricePoint(input_per_mtok=round(per_mtok, 4),
                      output_per_mtok=round(per_mtok, 4), amortised=True)
```

At \$2.80 per GPU-hour and 1,400 tokens per second, peak-throughput arithmetic says \$0.56 per million tokens. At a realistic 45% utilisation it is \$1.23—still competitive, but no longer the landslide the benchmark implied. Publish the utilisation assumption next to any self-hosting cost claim, or the claim is not checkable.

2.4 The Seven Properties of a Profile

Every profile carries seven groups of properties. Six are measurable; the seventh is a governance decision.

2.4.1 Context window

Two numbers, not one: total context and maximum output tokens. Routing needs both, because the gate is $\text{input} + \text{expected output} \leq \text{context}$, and a model with a 200k context but a 4k output cap cannot produce a long document.

The subtlety is that *effective* context is shorter than advertised context. Retrieval accuracy degrades well before the limit, and the degradation is position-dependent. Treat the advertised number as a hard ceiling and your own measured number as the routing threshold.

2.4.2 Tool calling and structured output

These are two capabilities that are often conflated. Tool calling is the ability to emit a well-formed call against a supplied schema. Structured output is the ability to constrain the *final* response to a schema. A model can have one without the other, and an extraction pipeline that needs guaranteed JSON is broken by a model that only has “JSON mode” as a best-effort instruction.

For routing, the practical distinctions are: does the provider enforce the schema at decode time (constrained decoding) or merely instruct the model to follow it; and does it support parallel calls, which changes agent latency substantially.

2.4.3 Quality

Quality is a *vector over task classes*, and it must be measured on your own evaluation set. Public leaderboard numbers are the wrong input for three reasons: they measure a distribution that is not yours, they are contaminated by training-data leakage in ways nobody can audit, and they are aggregates that hide exactly the per-class structure a router needs.

app/routing/schemas.py

```
# quality[task_class] in [0,1]; measured on YOUR eval set, never a
# leaderboard number. Missing key means "never measured" -- which the
# scorer treats as a prior, not as zero.
quality: dict[TaskClass, float] = Field(default_factory=dict)

def quality_for(self, task: TaskClass, prior: float = 0.55) -> float:
    """Measured quality, or a deliberately mediocre prior.

    Returning a prior rather than 0 keeps an unmeasured model routable
```

```
(so the bandit in ``adaptive.py`` can explore it) while making sure it
never beats a measured strong model on score alone.
"""
return self.quality.get(task, prior)
```

The prior is a small design decision with a large consequence. A missing measurement is not evidence of poor quality, so scoring it as zero makes a new model permanently unroutable and the catalog permanently stale. A mediocre prior plus a small explicit penalty keeps it eligible, lets bounded exploration measure it (Chapter 15), and stops it from winning by accident.

2.4.4 Cost

Cost has more dimensions than a price list suggests: input, output, cached input, and sometimes a separate rate for reasoning tokens. Output is typically three to five times input, which means *output length is the dominant cost variable*, not model choice, on verbose models. Cached input can be a quarter of the fresh rate or less, which makes prompt structure (Chapter 12) a bigger lever than most routing decisions.

2.4.5 Latency

Three numbers, each with a distribution:

TTFT Time to first token. Dominated by prefill, so it scales with *uncached* prompt length, and it is what the user perceives as responsiveness.

TPOT Time per output token in steady-state decode, also called inter-token latency (ITL). It scales with batch pressure on the serving replica, which is why a number measured off-peak is not predictive at peak.

Total TTFT + TPOT × (output tokens − 1). For a 500-token answer, TPOT dominates by an order of magnitude.

Record percentiles, not means. A mean TPOT hides the tail entirely, and the tail is where SLO breaches live.

```
app/routing/schemas.py
```

```
class LatencyProfile(BaseModel):
    """Measured, not advertised. Populated by the health prober."""
    ttft_p50_ms: float = 0.0
    ttft_p95_ms: float = 0.0
    tpot_p50_ms: float = 0.0          # inter-token latency, decode steady state
    tpot_p95_ms: float = 0.0
    samples: int = 0

    def estimate_ms(self, output_tokens: int, *, tail: bool = False) -> float:
        """End-to-end latency estimate for a generation of a given length."""
        ttft = self.ttft_p95_ms if tail else self.ttft_p50_ms
        tpot = self.tpot_p95_ms if tail else self.tpot_p50_ms
        return ttft + tpot * max(0, output_tokens - 1)
```

2.4.6 Throughput, reliability, availability

Throughput is tokens per second at a given concurrency, and it is a property of the deployment, not the model. Reliability is the rolling success rate of real traffic. Availability is the fraction of time the deployment is answering at all, and it differs from reliability in an important way: a provider can be 100% available and 60% reliable if it is returning 429s.

Rate limits belong here too, in both dimensions: requests per minute and tokens per minute. Tokens per minute is the one that binds in practice, and it cannot be enforced on arrival because output length is unknown—see Chapter 8.

2.4.7 Risk profile

The governance property: the maximum data classification this deployment is approved to receive. It is not derived from the model; it is derived from the contract, the region, the retention policy, and whether the provider trains on customer data. Chapter 14 develops the classification side; the registry’s job is to carry the answer and enforce it as a gate.

! Pitfall / Warning

The risk tier of a *provider* and of a *deployment* are different fields and both must be checked. A vendor approved for confidential data overall may have a specific region or a specific model tier under different terms. Checking only the provider is the mistake that produces a residency incident with a perfectly clean routing log.

2.5 Measuring the Profile Honestly

Everything above is worthless if the numbers come from a vendor’s marketing page. Table 2.4 is the measurement protocol.

Table 2.4: How to populate each field, and the mistake to avoid.

Field	How to measure	Common mistake
Quality	150–400 labelled examples per task class from <i>your</i> traffic, graded by a rubric with a human-agreement check on a sample.	Using a leaderboard; using one aggregate number.
Cost	Provider-reported usage from real requests, not token estimates.	Pricing on the mean output length; ignoring the p95.
Latency	Synthetic probes at fixed prompt and output lengths, plus passive observation of real traffic, both at percentiles.	Measuring off-peak; reporting means.
Throughput	A load test at your real concurrency and prompt-length distribution.	Using the vendor’s peak-throughput figure.
Reliability	Rolling success rate over a window, per deployment.	Aggregating across regions, which hides a single bad region.
Rate limits	Documented values, then the actual observed 429 boundary, which is frequently lower.	Trusting the documented number.
Risk tier	The contract and the DPA, signed off by whoever owns that risk.	An engineer’s judgement in a code comment.

✓ Best Practice

Re-measure latency continuously and quality quarterly. Latency drifts weekly as providers rebalance capacity and as your prompt lengths grow. Quality drifts when the provider silently updates a model behind a stable name— which is why the profile carries a `version` field and why your evaluation harness should alert on a quality delta with no version change.

2.6 Registry Architecture

Figure 2.1 shows the registry and the query it answers.

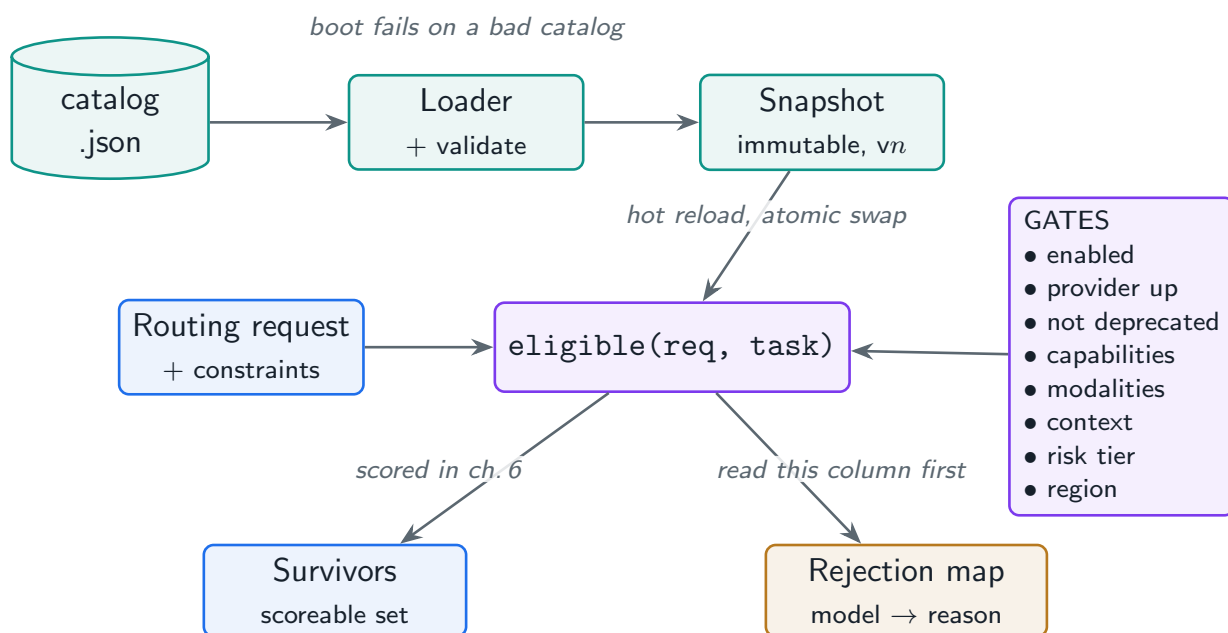


Figure 2.1: The registry as a queryable catalog. Reads are lock-free against an immutable snapshot; a hot reload swaps the snapshot atomically, so a deprecation is a data change rather than a deploy.

Three design decisions in that picture are worth defending.

The catalog is data. Model identifiers in source code are a deployment coupling you will regret at 2 a.m. on the day a provider deprecates something. Loading from JSON, with a hot reload endpoint, turns “take that model out of rotation” into a one-line change with no build.

Validation is fatal at boot. A dangling provider reference or a `max_output > context_window` is a configuration error that will otherwise surface as a routing failure under load. Fail at startup, where someone is watching.

Reads are lock-free. Routing happens on every request; a reader lock on that path shows up as p99 latency. An immutable snapshot swapped atomically gives correct reads with no contention.

`app/routing/registry.py`

```

class ModelRegistry:
    """Thread-safe, hot-reloadable catalog of deployments."""

    def eligible(self, req: RoutingRequest, task: TaskClass,

```

```

    ) -> tuple[list[ModelProfile], dict[str, str]]:
        """Hard filtering. Returns (survivors, {model_id: rejection reason}).

        Ordering matters for the operator reading the rejection map: cheap,
        deterministic checks first, so the reason surfaced is the most
        fundamental one rather than an incidental downstream failure.
        """
        c = req.constraints
        survivors: list[ModelProfile] = []
        rejected: dict[str, str] = {}
        now = datetime.now(timezone.utc)

        if c.pinned_model:
            # audited escape hatch
            m = self._models.get(c.pinned_model)
            if m is None:
                raise KeyError(f"pinned model not in registry: {c.pinned_model}")
            return [m], {}

        for m in self._models.values():
            if not m.enabled:
                rejected[m.id] = "disabled"
                continue
            prov = self._providers.get(m.provider)
            if prov is None or not prov.enabled:
                rejected[m.id] = f"provider {m.provider} disabled"
                continue
            if m.deprecated_after and m.deprecated_after <= now:
                rejected[m.id] = f"deprecated {m.deprecated_after:%Y-%m-%d}"
                continue

            # Capability gates -- binary, never scored.
            missing = c.required_capabilities - m.capabilities
            if missing:
                rejected[m.id] = f"missing {' '.join(sorted(x.value for x in
                    ↪ missing))}"
                continue
            need_mod = c.required_modalities | set(req.attachments)
            if not need_mod <= set(m.modalities_in):
                lack = need_mod - set(m.modalities_in)
                rejected[m.id] = f"no input modality {' '.join(x.value for x in lack)}"
                continue
            if m.context_window < c.min_context_window:
                rejected[m.id] = (f"context {m.context_window} < required "
                    f"{c.min_context_window}")
                continue

            # Governance gates. The tier is derived from the payload in
            # security.py, never from what the caller claims.
            if _RISK_ORDER[m.max_risk_tier] < _RISK_ORDER[c.risk_tier]:
                rejected[m.id] = (f"risk tier {m.max_risk_tier.value} < required "
                    f"{c.risk_tier.value}")
                continue
            if _RISK_ORDER[prov.max_risk_tier] < _RISK_ORDER[c.risk_tier]:
                rejected[m.id] = f"provider trust tier below {c.risk_tier.value}"

```



```

        continue
    if c.allowed_regions is not None:
        if not (set(m.regions) & set(c.allowed_regions)) and \
            "global" not in m.regions:
            rejected[m.id] = f"no deployment in {c.allowed_regions}"
            continue

    survivors.append(m)

return survivors, rejected

```

2.6.1 The catalog format

app/routing/catalog.json (excerpt)

```

{
  "providers": [
    {
      "id": "cluster_eu",
      "display_name": "Self-hosted vLLM cluster (eu-west)",
      "kind": "self_hosted",
      "base_url": "http://vllm.eu-west.internal:8000",
      "regions": ["eu-west"],
      "data_retention_days": 0,
      "trains_on_customer_data": false,
      "max_risk_tier": "restricted"
    }
  ],
  "models": [
    {
      "id": "cluster_eu/mistral-24b",
      "family": "mistral-24b-fp8",
      "provider": "cluster_eu",
      "version": "2026-05",
      "modalities_in": ["text"],
      "capabilities": ["streaming", "json_mode", "tool_calling",
        "structured_output"],
      "context_window": 65536,
      "max_output_tokens": 8192,
      "price": { "input_per_mtok": 0.11, "output_per_mtok": 0.11,
        "amortised": true },
      "latency": { "ttft_p50_ms": 210, "ttft_p95_ms": 640,
        "tpot_p50_ms": 14, "tpot_p95_ms": 38, "samples": 500 },
      "quality": { "classification": 0.84, "extraction": 0.83,
        "summarisation": 0.81, "rag_qa": 0.79,
        "code_generation": 0.72, "planning": 0.67 },
      "reliability": 0.985,
      "regions": ["eu-west"],
      "max_risk_tier": "restricted",
      "tags": ["open-weight", "self-hosted", "tier-medium", "eu-resident"]
    }
  ]
}

```

The `tags` field carries the tier membership that the difficulty band uses (Chapter 4) and the policy labels that the tier and residency rules select on (Chapter 3). Expressing policy in terms of